GUÍA Completa Fases de desarrollo app

Caso 5 · Plataforma educativa

La academia quiere desarrollar una plataforma educativa para:

- gestionar alumnos y profesores;
- crear cursos;
- publicar contenidos;
- realizar ejercicios;
- consultar el progreso;
- enviar notificaciones.

En la **Parte 1** ya hemos realizado el **análisis de requisitos**.

Ahora debemos transformar esos requisitos en una solución software.

Para ello trabajaremos las siguientes fases:

Diseño → Implementación → Pruebas → Explotación → Mantenimiento

La forma de organizar estas fases dependerá del **modelo de ciclo de vida elegido**.

- En un modelo **secuencial**, las fases se realizan principalmente de forma ordenada.
- En un modelo **incremental o iterativo**, algunas fases se repiten para cada incremento o iteración.
- En **Scrum**, el trabajo se organiza en Sprints y, para conseguir cada incremento, pueden realizarse actividades de diseño, implementación y pruebas. La explotación y el mantenimiento dependerán de cuándo se pongan las funcionalidades en producción.

Por tanto, debemos distinguir entre:

> **Las actividades necesarias para desarrollar y mantener el software**

y

> **La forma en que el modelo de ciclo de vida organiza esas actividades.**

PARTE 1 · 📋 ANÁLISIS DE REQUISITOS

> **Esta parte se ha realizado previamente.**

Los requisitos nos indican **qué necesita el cliente y qué debe hacer el sistema**.

Ejemplos:

- El sistema permitirá registrar alumnos.
- Los profesores podrán crear cursos.
- Los profesores podrán publicar contenidos.
- Los alumnos podrán realizar ejercicios.
- Los alumnos podrán consultar su progreso.

Resultado de esta fase

Al terminar el análisis debemos saber:

> **¿Qué debe hacer la aplicación?**

Pero todavía no sabemos exactamente:

> **¿Cómo vamos a construirla?**

Para eso necesitamos el diseño.

PARTE 2 · 🏗️ DISEÑO DE LA SOLUCIÓN

¿Qué hacemos?

En el diseño decidimos **cómo estará organizada la aplicación**.

No tenemos que programar.

Debemos identificar:

- entidades;
- relaciones;
- módulos;
- pantallas;
- arquitectura.

2.1 Entidades

Una entidad representa algo sobre lo que el sistema necesita guardar información.

Ejemplo

Entidad	¿Qué información puede almacenar?
Alumno	nombre, email, contraseña

Profesor	nombre, email
Curso	nombre, descripción
Contenido	título, texto, tipo
Ejercicio	preguntas, respuestas
Progreso	alumno, curso, porcentaje

✏️ Actividad

Identifica **5 entidades** de la plataforma.

ID	Entidad
E-01	Alumno
E-02	Profesor
E-03	Curso
E-04	Contenido
E-05	Ejercicio

2.2 Relaciones

Las entidades se relacionan entre sí.

Ejemplo:

```
ALUMNO
  |
  | se matricula
  ↓
CURSO
```

Otro ejemplo:

```
PROFESOR
  |
  | crea
  ↓
CURSO
```

✏️ Actividad

Identifica **3 relaciones**.

Entidad	Relación	Entidad
Profesor	Realiza	Curso
Alumno	Realiza	Ejercicio
Ejercicio	Contiene	Contenido

2.3 Módulos

Un módulo agrupa funcionalidades relacionadas.

Ejemplo:

Módulo	Funcionalidades
Usuarios	Registro, login
Cursos	Crear, consultar cursos
Contenidos	Publicar, consultar
Ejercicios	Crear y realizar ejercicios
Seguimiento	Consultar progreso
Notificaciones	Enviar avisos

✏️ Actividad

Identifica **4 módulos** para la plataforma.

2.4 Pantallas

Debemos identificar las principales pantallas de la aplicación.

Pantalla	Usuario	Función
Login	Alumno / profesor	Acceder
Mis cursos	Alumno	Consultar cursos

Gestión de cursos	Profesor	Crear cursos
Contenidos	Alumno	Consultar contenidos
Ejercicios	Alumno	Realizar ejercicios

✏️ Actividad

Propón **4 pantallas** para la aplicación.

→ Pantalla1

1. Login

2. Registro

3. Salir

→Pantalla2 Alumno

1. Contenidos del curso

2. Ejercicios

3. Consultar seguimiento

4. Volver atrás

→ Pantalla3 Profesor

1. Crear curso

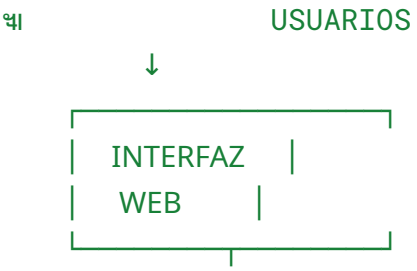
2. Crear ejercicios

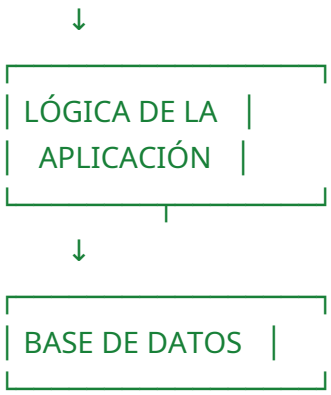
3. Pasar lista

4. Salir

2.5 Arquitectura

Representamos de forma sencilla cómo se organiza técnicamente la aplicación.





Debemos entender:

Componente	Función
Interfaz	Permite utilizar la aplicación
Lógica	Gestiona las operaciones y reglas
Base de datos	Almacena la información

PARTE 3 · 💻 IMPLEMENTACIÓN

¿Qué hacemos?

En esta fase **construimos la solución diseñada**.

Los Developers escriben el código y crean los componentes necesarios.

Por ejemplo:

Requisito

> El alumno puede iniciar sesión.

Diseño

- pantalla de login;
- usuario;
- contraseña;
- sistema de autenticación.

Implementación

Se programa todo lo necesario para que esa funcionalidad funcione.

3.1 Implementación progresiva

En nuestro caso podemos dividir el desarrollo.

Sprint 1

- registro;
- inicio de sesión.

Sprint 2

- crear cursos;
- consultar cursos.

Sprint 3

- publicar contenidos;
- consultar contenidos.

Idea clave

> **No tenemos que construir toda la aplicación de una sola vez.**

En Scrum construimos el producto progresivamente mediante incrementos.

PARTE 4 · 🧪 PRUEBAS

¿Qué hacemos?

Comprobamos que:

- la aplicación funciona;
- cumple los requisitos;
- los datos se procesan correctamente;
- no existen errores importantes.

4.1 Plan de pruebas

Para cada requisito importante podemos definir una prueba.

ID	Requisito	Prueba	Resultado esperado

CP-01	Registrar alumno	Introducir datos válidos	Alumno registrado
CP-02	Iniciar sesión	Introducir usuario y contraseña correctos	Acceso permitido
CP-03	Crear curso	Introducir los datos del curso	Curso creado
CP-04	Consultar curso	Seleccionar un curso	Se muestra el curso
CP-05	Realizar ejercicio	Responder a un ejercicio	Se guarda el resultado

✏️ Actividad

Crea **3 casos de prueba** para funcionalidades de la plataforma.

4.2 ¿Qué ocurre si una prueba falla?

```
PRUEBA
  ↓
ERROR
  ↓
ANALIZAR
  ↓
CORREGIR
  ↓
VOLVER A PROBAR
```

Por ejemplo:

> El alumno introduce correctamente sus datos, pero el sistema no permite iniciar sesión.

Hay que corregir el problema y volver a realizar la prueba.

PARTE 5 · 🚀 EXPLOTACIÓN

¿Qué significa?

La explotación comienza cuando la aplicación se pone en funcionamiento en un **entorno real**.

En nuestro caso:

> La academia pone la plataforma a disposición de alumnos y profesores.

```
DESARROLLO
  ↓
PRUEBAS
  ↓
DESPLIEGUE
  ↓
USUARIOS REALES
  ↓
EXPLOTACIÓN
```

5.1 Antes de ponerla en funcionamiento

Debemos comprobar, por ejemplo:

Aspecto	¿Qué comprobamos?
Instalación	¿La aplicación está correctamente desplegada?
Acceso	¿Los usuarios pueden entrar?
Datos	¿La información está correctamente configurada?
Seguridad	¿Los usuarios están protegidos?
Rendimiento	¿La aplicación responde correctamente?
Copias de seguridad	¿Podemos recuperar los datos?

✏️ Actividad

Indica **tres comprobaciones** que realizarías antes de poner la plataforma en funcionamiento.

PARTE 6 · 🔧 MANTENIMIENTO

¿Termina el proyecto cuando se entrega?

No.

Una aplicación continúa evolucionando durante su vida útil.

Podemos encontrar diferentes tipos de mantenimiento.

6.1 Mantenimiento correctivo

Soluciona **errores**.

> El sistema no permite iniciar sesión.

6.2 Mantenimiento adaptativo

Adapta la aplicación a **cambios del entorno**.

> La plataforma debe funcionar con una nueva tecnología o entorno.

6.3 Mantenimiento evolutivo

Añade **nuevas funcionalidades**.

> Los profesores quieren crear cuestionarios automáticos.

6.4 Mantenimiento perfectivo

Mejora una funcionalidad que ya existe.

> La pantalla de cursos resulta complicada y se rediseña para facilitar su uso.

6.5 Actividad

Identifica el tipo de mantenimiento:

Situación	Tipo de mantenimiento

No funciona el inicio de sesión	Correctivo
Se añaden cuestionarios automáticos	Evolutivo
Se adapta la aplicación a un nuevo entorno tecnológico	Adaptativo
Se mejora la pantalla de cursos	Perfectivo

PARTE 7 · 🔄 ¿CÓMO ENCAJA TODO CON SCRUM?

Las fases anteriores **siguen existiendo aunque utilicemos Scrum**.

La diferencia es que no tenemos necesariamente que hacer todo el diseño, toda la implementación y todas las pruebas antes de entregar algo.

Podemos trabajar así:



Ejemplo

Sprint 1

Diseñamos, implementamos y probamos el registro y el login.

→ Obtenemos un incremento.

Sprint 2

Diseñamos, implementamos y probamos la gestión de cursos.

→ Obtenemos un nuevo incremento.

Sprint 3

Diseñamos, implementamos y probamos los contenidos.

→ Obtenemos otro incremento.

🧠 PARTE 8 · RESUMEN

El proceso completo que debe conocer el alumno es:

Fase	Pregunta que responde
1. Análisis de requisitos	¿Qué necesita el usuario?
2. Diseño	¿Cómo organizaremos la solución?
3. Implementación	¿Cómo construiremos la solución?
4. Pruebas	¿Funciona correctamente?
5. Explotación	¿Cómo ponemos la aplicación en funcionamiento?
6. Mantenimiento	¿Cómo corregimos y evolucionamos la aplicación?

Y en Scrum:

Estas actividades pueden repetirse en diferentes Sprints para construir progresivamente el producto.

```
↯REQUISITOS
  ↓
DISEÑO
  ↓
IMPLEMENTACIÓN
```

```
  ↓
PRUEBAS
  ↓
INCREMENTO
  ↓
EXPLOTACIÓN / USO
  ↓
FEEDBACK
  ↓
NUEVOS REQUISITOS
  ↓
NUEVO SPRINT
```

↯**Idea fundamental:** el alumno debe distinguir entre **las actividades necesarias para desarrollar software** (diseñar, implementar, probar, explotar y mantener) y **el modelo utilizado para organizar el desarrollo** (por ejemplo, Scrum, Kanban, incremental o espiral)

📊 Rúbrica de evaluación — Caso 5: Plataforma educativa

Puntuación total: 10 puntos

Criterio	Excelente	Adecuado	En proceso	Insuficiente	Puntos
1. Análisis de requisitos	Identifica correctamente los requisitos funcionales y los formula de forma clara y precisa.	Identifica la mayoría correctamente, con pequeños errores.	Identifica algunos, pero son incompletos o poco precisos.	No identifica adecuadamente las necesidades del sistema.	**1,5**
2. Diseño de la solución	Identifica y relaciona correctamente entidades, relaciones, módulos, pantallas y arquitectura. Las decisiones son	Identifica los principales elementos del diseño y existe coherencia general.	El diseño es incompleto o presenta algunas relaciones o elementos incorrectos.	No consigue representar una solución coherente.	**2,0**

	coherentes con los requisitos.				
3. Implementación	Explica correctamente cómo se transformarían los diseños en funcionalidades y organiza su desarrollo de forma coherente.	Explica adecuadamente la implementación, aunque con poco detalle.	Comprende parcialmente la relación entre diseño e implementación.	Confunde implementación con otras fases o no explica cómo se construiría la solución.	**1,0**
4. Plan de pruebas	Diseña pruebas claras y relacionadas con los requisitos, indicando correctamente el resultado esperado.	Propone pruebas adecuadas para los principales requisitos.	Las pruebas son incompletas o poco relacionadas con los requisitos.	No sabe plantear pruebas adecuadas.	**1,5**
5. Explotación	Explica correctamente qué implica poner el sistema en funcionamiento y propone comprobaciones relevantes antes de su puesta en producción.	Comprende la finalidad de la explotación y propone algunas comprobaciones.	La explicación es parcial o confunde explotación con desarrollo.	No comprende qué significa poner el sistema en explotación.	**1,0**
6. Mantenimiento	Distingue correctamente mantenimiento correctivo, adaptativo, evolutivo y perfectivo y los aplica	Distingue la mayoría de los tipos y los aplica adecuadamente.	Confunde algunos tipos de mantenimiento.	No distingue los tipos de mantenimiento.	**1,0**

	correctamente al caso.				
7. Relación con el ciclo de vida elegido	Comprende que las fases anteriores pueden organizarse de diferente manera según el modelo y explica correctamente su aplicación a Scrum u otro modelo.	Comprende la relación general entre fases y modelo de ciclo de vida.	Presenta cierta confusión entre fases y modelo.	Considera que todos los modelos organizan las fases de la misma manera.	**1,0**
8. Coherencia global y aplicación al caso	Todas las decisiones están relacionadas entre sí y forman una solución coherente para la plataforma educativa.	La solución es globalmente coherente.	Existen contradicciones o decisiones poco justificadas.	Las respuestas aparecen desconectadas del caso.	**1,0**

Total: 10 puntos

🎯 Niveles de logro

Puede resultar útil traducir la puntuación final a cuatro niveles:

Puntuación	Nivel	Interpretación
9 – 10	🟢 Excelente	Comprende las fases y sabe aplicarlas correctamente al caso.
7 – 8,9	🟢 Adecuado	Comprende las fases y las aplica con pequeños errores.
5 – 6,9	🟡 En proceso	Comprende parte de los conceptos, pero necesita mejorar su aplicación.

0 – 4,9	🔴 Insuficiente	No comprende todavía las fases o no consigue aplicarlas al caso.

💡 Criterio especialmente importante

No debería penalizarse al alumno por elegir, por ejemplo, **una arquitectura o distribución de módulos diferente** a la propuesta en la solución, siempre que:

1. sea técnicamente razonable;
2. responda a los requisitos;
3. esté correctamente explicada.

La rúbrica evalúa principalmente **la capacidad de analizar y justificar una solución**, no la coincidencia literal con una respuesta modelo.